

Architecture - Revision 2

1. Architectural Decision

Phase 1 will use a **SQLite-backed deterministic watchdog**. JSON registry design is rejected for Phase 1 due to concurrency and corruption risk.

2. Component Overview

OpenClaw agent

- > registration interface
- > SQLite task registry
- > scheduler (cron/systemd timer)
- > watchdog evaluator
- > notification emitter
- > evidence/logging

3. Registration Interface

Phase 1 requires a concrete registration entry point. Recommended pattern:

- `agent-follow-up register --id ... --title ... --interval ... --target ... --observable-type ... --observable-id ...`

The registration interface writes a task row and initial event row into SQLite.

4. Observation Strategy by Task Type

Task Type	Observation Source	Phase 1 Support
Local process-backed	PID / process status / exit code	Yes
Exec/background session-backed	OpenClaw process/session identifier if concretely queryable	Conditional
OpenClaw delegated session without reliable identifier	Insufficiently defined	No (Phase 2)
Manual/external marker-backed	Known marker file, DB row, or explicit completion signal	Yes

5. Notification Emitter

Revision 2 requires one concrete emitter mechanism. Recommended design direction:

- watchdog invokes a local OpenClaw-native send path with target chat/session id and a deterministic templated message
- delivery attempt and result are logged in SQLite

If this mechanism cannot be concretely documented against available OpenClaw interfaces, implementation must not begin.

6. Unknown-State Boundary

A task becomes **unknown** when:

- it was previously observable,

- its expected observation source no longer confirms running,
- no terminal success/failure condition is proven,
- and this condition persists across a defined number of watchdog cycles.